| | |
|---|---|
| **Stato** | Completato |
| **Iniziato** | giovedì, 19 febbraio 2026, 11:36 |
| **Terminato** | giovedì, 19 febbraio 2026, 11:48 |
| **Tempo impiegato** | 12 min. 17 secondi |

**Domanda 1**

Parzialmente corretta

Hai avviato un container da un'immagine `ubuntu`. Mentre il container è in esecuzione, crei un file `/tmp/test.txt` al suo interno. Poi fermi il container con `docker stop`. Qual è lo stato del sistema?

a. Il file `/tmp/test.txt` viene perso non appena il container si ferma

b. Se avvii un *nuovo* container dalla stessa immagine (`docker run ubuntu`), questo non conterrà il file `/tmp/test.txt` ✓

c. L'immagine originale `ubuntu` è stata modificata per includere il nuovo file.

d. Il file esiste ancora nel "container layer" scrivibile. Se riavvii lo stesso container (`docker start`), il file sarà lì. ✓

Le risposte corrette sono: Il file esiste ancora nel "container layer" scrivibile. Se riavvii lo stesso container (`docker start`), il file sarà lì., Se avvii un *nuovo* container dalla stessa immagine (`docker run ubuntu`), questo non conterrà il file `/tmp/test.txt`

**Domanda 2**

Risposta errata

In uno scenario open-source con fork, quali ruoli hanno tipicamente `origin` e `upstream`?

a. `upstream` è il tuo fork

b. `upstream` è il repository originale ✓

c. `origin` è il repository originale

d. `origin` è il tuo fork ✓

Le risposte corrette sono: `origin` è il tuo fork, `upstream` è il repository originale

**Domanda 3**

Risposta errata

Supponendo che `list` sia un'istanza di `List<int>`, come fa il compilatore C# a risolvere la chiamata `list.Where(x => x > 5)`, visto che `Where` non è un metodo definito nella classe `List<T>`?

a. Cerca nelle sottoclassi di `List<T>` delle definizioni di metodi, chiamati `Where`, il cui primo parametro sia marcato con `this` e abbia un tipo compatibile con quello di `list`

b. `Where` è un metodo virtuale di `System.Object` che viene ereditato da tutte le classi

c. Il compilatore genera del codice che sfrutta la reflection, a runtime, per trovare metodi appropriati

d. Cerca nelle classi statiche le definizioni di metodi statici, chiamati `Where`, il cui primo parametro sia marcato con `this` e abbia un tipo compatibile con quello di `list` ✓

La risposta corretta è: Cerca nelle classi statiche le definizioni di metodi statici, chiamati `Where`, il cui primo parametro sia marcato con `this` e abbia un tipo compatibile con quello di `list`

**Domanda 4**

Risposta errata

Perché alcuni Dockerfile contengono più istruzioni `FROM`, per esempio:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
...
FROM mcr.microsoft.com/dotnet/runtime:9.0
COPY --from=build ...
```

Qual è lo scopo di questa tecnica?

a. Velocizzare il download dell'immagine permettendo il download parallelo dei layer

b. Ridurre la dimensione dell'immagine finale scartando gli strumenti di compilazione (SDK) e mantenendo solo il runtime e i binari compilati ✓

c. Permettere al container di eseguire sia codice .NET 9 che codice legacy

d. Creare due immagini distinte con un solo comando `docker build`

La risposta corretta è: Ridurre la dimensione dell'immagine finale scartando gli strumenti di compilazione (SDK) e mantenendo solo il runtime e i binari compilati

**Domanda 5**

Risposta corretta

Quali operazioni sono tipicamente possibili via reflection?

a. Invocare metodi con `MethodInfo.Invoke(...)` ✓
b. Creare nuove istanze con `new`
c. Creare istanze con `Activator.CreateInstance(...)` ✓
d. Generare automaticamente unit-test
e. Caricare assembly e scoprire tipi a runtime (plugin) ✓

Le risposte corrette sono: Creare istanze con `Activator.CreateInstance(...)`, Invocare metodi con `MethodInfo.Invoke(...)`, Caricare assembly e scoprire tipi a runtime (plugin)

**Domanda 6**

Risposta corretta

In un workflow GitHub Actions, hai tre job: `build`, `test` e `deploy`, definiti dalla configurazione seguente:

```
jobs:
  build:
    ...
  test:
    needs: build
    ...
  deploy:
    needs: build
    ...
```

Quali job verranno eseguiti e in che ordine?

a. `build` parte per primo. Se finisce con successo, `test` e `deploy` vanno in parallelo ✓
b. `build` parte per primo. Se finisce con successo, partirà `test`. Se finisce con successo, verrà eseguito `deploy`
c. Sequenziale: `build`, poi `test`, poi `deploy`
d. `build`, `test` e `deploy` partono tutti contemporaneamente
e. Se `build` fallisce, `test` e `deploy` non verranno eseguiti ✓

Le risposte corrette sono: `build` parte per primo. Se finisce con successo, `test` e `deploy` vanno in parallelo, Se `build` fallisce, `test` e `deploy` non verranno eseguiti

**Domanda 7**

Risposta corretta

Perché esiste `IDisposable` in .NET?

a. Ragioni storiche/di compatibilità: da C# 8 non serve più
b. Serve per rilasciare deterministicamente risorse non gestite che sono incapsulate da oggetti .NET ✓
c. Per permettere agli sviluppatori di creare nuove classi

La risposta corretta è: Serve per rilasciare deterministicamente risorse non gestite che sono incapsulate da oggetti .NET

**Domanda 8**

Parzialmente corretta

Quali affermazioni sono vere riguardo la *staging area (index)* in Git?

a. Funziona solo in modalità online

b. È obbligatoria solo nei repository molto grandi

c. È un backup automatico su GitHub

d. Consente di costruire commit "pezzo per pezzo" ✓

e. Permette di selezionare quali modifiche includere nel prossimo commit ✓

Le risposte corrette sono: Permette di selezionare quali modifiche includere nel prossimo commit, Consente di costruire commit "pezzo per pezzo"

Fine revisione